CLOUD COMPUTING PROJECT REPORT

Cloud-Based Knowledge Graph Framework for Automated Research Intelligence

Submitted by: Dhruv Gupta (23BIT0350), Parth Sharma (23BIT0211), Devesh Agarwal (23BIT0201)

Programme / Branch / Year: B.Tech. Information Technology, IV Year

Course Code and Name: BITE412L, Cloud Computing

Institution: School of Computer Science and Engineering, Vellore Institute of Technology, Vellore

Course Instructor / Guide: Dr. Priya V

Team Number:

Cloud Provider: Amazon Web Services (AWS), with Neo4j AuraDB, Groq and Google Gemini as external managed services

Git Repository URL:

https://github.com/DeveshAg30/Cloud_Based_Knowledge_Graph_Framework_for_Automated_Research_Intelligence_Aws_Project_2026

Date of Submission: 16/10/2026

Abstract

Researchers and R&D teams cannot keep pace with the volume of scientific literature. Keyword search engines miss relationships that span several papers, and standalone large language models (LLMs) hallucinate facts and cannot cite their sources. This project built a cloud-based knowledge graph framework that reads scientific papers automatically, extracts their authors, institutions, methods, tasks, datasets and metrics together with the relations between them, and answers natural-language questions only from facts retrieved from the graph (GraphRAG), citing the source papers. The system runs on Amazon Web Services. Papers land in an Amazon S3 data lake; an S3 event triggers an AWS Lambda function; two fine-tuned SciBERT models served from an Amazon SageMaker serverless endpoint extract entities and relations; and the results are written to an openCypher knowledge graph. Researchers sign in through Amazon Cognito and use a dashboard hosted on AWS Amplify that calls Amazon API Gateway and a second Lambda function. Amazon CloudWatch, AWS X-Ray and Amazon SNS provide monitoring and alerting. Because the AWS Free plan blocks Amazon Neptune and Amazon Bedrock, the deployed graph runs on Neo4j AuraDB Free and the LLM layer on Groq and Gemini; one deployment parameter switches each back to the AWS service. Measured results: entity recognition F1 of 0.618 and relation extraction F1 of 0.661 on the SciERC test set; a graph of 29,320 nodes and 42,015 relationships built from 306 ingested documents with no failed documents; and, on a 28-question benchmark, 0% hallucinated answers with GraphRAG against 62.5% for the same LLM without the graph, with 98.3% citation precision. All 13 end-to-end checks on the deployed system pass. The project shows that grounding an LLM in an automatically built knowledge graph makes literature question answering verifiable, at near-zero cloud cost.

Keywords

Knowledge graph; GraphRAG; SciBERT; relation extraction; serverless computing; Amazon Web Services.

Table of Contents

Abstract 2

Keywords 2

Table of Contents 3

1. Introduction 6

1.1 Background 6

1.2 Problem Statement 6

1.3 Motivation and Significance 6

1.4 Scope 7

1.5 Report Organisation 7

2. Literature Survey / Related Work 7

2.1 Comparative Synthesis 10

2.2 Individual Research Gap Analysis 10

2.3 Consolidated Research Gap 12

3. Project Objectives 12

3.1 Expected Outcomes 13

4. Novelty Summary 13

5. Proposed System and Architecture 14

5.1 System Overview 14

5.2 Diagram 1 - Cloud Deployment Architecture 15

5.3 Diagram 2 - Complete System Architecture / Workflow 16

5.4 Workflow and Use Cases 16

5.5 Security, Availability and Scalability 17

6. Dataset / Input Data Description 18

6.1 Data Preparation 19

7. Cloud Platform and Services Planning 20

7.1 Deployment and Network Design 20

7.2 Cost Estimation 21
7.3 Portability / Provider-Specific Choices 21
8. Methodology and Implementation Plan 22
8.1 Overall Methodology 22
8.2 Tools and Technologies 22
8.3 Step-by-Step Implementation Plan 22
8.4 Evaluation Metrics 23
9. Implementation / Development 23
9.1 Implemented Modules 24
9.2 Screenshots and Configuration Evidence 24
9.3 Repository and Project Structure 26
9.4 Version Control Workflow 27
10. Testing, Results and Observations 27
10.1 Test Plan and Test Cases 27
10.2 Results 28
10.3 Baseline Comparison 31
10.4 Results Summary 31
11. Discussion, Limitations and Risk Analysis 31
11.1 Discussion 31
11.2 Limitations 32
11.3 Risks and Mitigation 33
12. Team Contribution and Project Management 33
12.1 Individual Contribution Evidence 34
13. Conclusion and Future Enhancements 35
13.1 Conclusion 35
13.2 Future Enhancements 35
References 36

Appendix A - Project Timeline 38

Appendix B - Final Review Checklist 39

1. Introduction

1.1 Background

Scientific output grows every year: OpenAlex alone indexes more than 300 million scholarly works. Knowledge about which methods solve which tasks, which datasets they are evaluated on and who works with whom is spread across the text of these papers. A knowledge graph stores this knowledge as typed nodes (papers, authors, methods, datasets) and typed edges (AUTHORED, CITES, APPLIED_TO, EVALUATED_ON), so that multi-hop questions can be answered by traversing the graph.

Cloud computing suits this problem for four reasons. Ingestion is bursty: thousands of papers may arrive at once and then nothing for days, which matches serverless functions that scale to zero. The NLP models need more memory than a laptop session but are invoked only occasionally, which matches serverless inference. A managed graph database avoids operating database servers. Finally, researchers need a shared, always-available web application with authentication, which managed hosting and identity services provide.

1.2 Problem Statement

Researchers, students and enterprise R&D teams need to answer questions such as "Which methods have been applied to link prediction, and on which datasets?" or "How is TransE connected to BERT?". Today this means reading many papers by hand. Keyword-based digital libraries return documents, not answers, and cannot follow relations across documents. Standalone LLMs give fluent answers but invent authors, venues and results, and cannot point to a source. The problem addressed is therefore: **automatically turn unstructured scientific text into a queryable knowledge graph on the cloud, and answer natural-language questions from that graph with verifiable citations.**

1.3 Motivation and Significance

An answer that cites the exact papers it came from can be checked in seconds, which is what academic and corporate research requires. Automating extraction removes the manual bottleneck of literature reviews, and the graph exposes connections (shared methods, multi-hop paths, collaborating institutions) that keyword search cannot show. Running the system on serverless cloud services keeps the cost close to zero when idle, which makes it practical for a university group.

1.4 Scope

Included: ingestion of OpenAlex and S2ORC records and of PDF uploads; SciBERT entity recognition and relation extraction on Amazon SageMaker; an openCypher knowledge graph; GraphRAG question answering with citations; graph exploration, multi-hop connection search and entity profiles (Research 360) on a web dashboard; authentication, monitoring, alarms and an automated end-to-end test.

Excluded: full-text processing of the complete S2ORC and OpenAlex snapshots (hundreds of gigabytes); optical character recognition of scanned PDFs; multilingual papers; GPU training on AWS (the account's SageMaker training quota is zero, so models were trained locally and served on SageMaker); and production use of Amazon Neptune and Amazon Bedrock, which the AWS Free plan blocks (both are implemented as deployment options).

1.5 Report Organisation

Section 2 reviews 15 papers and Section 3 states the objectives. Section 4 explains the novelty, Section 5 the architecture and Section 6 the data. Section 7 covers the cloud services, network design and cost, and Section 8 the methodology. Section 9 describes the implementation, Section 10 the tests and measured results, and Section 11 the limitations and risks. Section 12 records team contributions and Section 13 concludes.

2. Literature Survey / Related Work

Fifteen papers were reviewed: twelve from IEEE, ACM, Springer, Elsevier, MDPI and other indexed venues and three recent arXiv preprints. Eleven were published in 2024-2026. Table 1 summarises each paper; the full APA references are listed at the end of the report.

Note on changes from the proposal: the proposal's survey included a paper on machine learning and cloud-based knowledge graphs for recognising suicidal tendencies, which has been **retracted** by its publisher, and a paper on automated knowledge graph construction for research information systems whose bibliographic record could not be verified in Crossref or other indexes. Both were replaced with verified, highly relevant papers: P4 (IEEE TKDE) and P9 (ACM TOIS).

Table 1. Summary of the 15 reviewed papers

ID	Citation / DOI	Problem addressed	Method / architecture	Dataset / evaluation	Key findings
P1	Nagy, Abonyi & Ruppert (2024), Applied Sciences 14(8), 3398. doi:10.3390/app14083398	Modelling operator-related factors in human-centred collaborative manufacturing (Industry 5.0)	Human-centric knowledge graph built on existing ontologies and standards (ISA-95); SPARQL-style semantic querying	Industrial wire-harness assembly case study (positioning and sensor data); qualitative demonstration	Shows that a KG can integrate movement, ergonomics and robot-collaboration data and answer multi-hop queries
P2	Yang (2025), Int. J. of Computer Science and Information Technology 7(1), 1-8. doi:10.62051/ijcsit.v7n1.01	Manual, slow construction of knowledge graphs	Review of deep-learning KG construction: CNNs, sequence models, GNNs and pre-trained Transformers for extraction, fusion and reasoning	Literature-based analysis with reported benchmark results	Deep models improve the efficiency and accuracy of extraction and fusion; consistency across heterogeneous data remains hard
P3	Xu, Zhang, Zhang, Lu & Xiao (2024), IEEE Access 12, 162638-162650. doi:10.1109/ACCESS.2024.3485877	LLM factual hallucination in a knowledge-intensive domain (traditional folklore)	Folklore ontology, zero-shot LLM triplet extraction (ZFTE), two-stage KG retrieval-augmented generation (TFKG-RAG)	Self-built TFCQD question set; accuracy	96.7% accuracy on TFCQD, better than the baselines tested; KG grounding reduces hallucination without fine-tuning the LLM
P4	Pan, Luo, Wang, Chen, Wang & Wu (2024), IEEE Trans. Knowledge and Data Engineering 36(7), 3580-3599. doi:10.1109/TKDE.2024.3352100	How LLMs and knowledge graphs can complement each other	Roadmap of three frameworks: KG-enhanced LLMs, LLM-augmented KGs, and synergised LLM+KG	Survey of published methods and benchmarks	KGs supply explicit, verifiable knowledge that LLMs lack; LLMs can help construct and query KGs
P5	Mohamed et al. (2025), arXiv:2502.15689	Fragmented research data across disciplines	KG construction with LLMs and NLP tools (spaCy), clustering, and embedding-based link prediction	Wikipedia sentences; FB15k-237 and WN18RR link-prediction benchmarks	KGs can connect fragmented research data and surface interdisciplinary links; quality needs automated extraction plus human review
P6	Napoli, Morabito, Celesti, Villari & Fazio (2026), arXiv:2602.07614	Decision-making in distributed (edge/IoT) systems without a central controller	Knowledge Sharing paradigm: local KGs plus GraphSAGE embedding exchange, producing a global Knowledge Map	Simulated resource orchestration over several network topologies	Keeps semantic coherence without central control; topology strongly affects semantic drift
P7	Peng, Xia, Naseriparsa & Osborne (2023), Artificial Intelligence Review 56(11), 13071-13102. doi:10.1007/s10462-023-10465-9	Overview of KG opportunities and open technical challenges	Systematic survey: embeddings, acquisition, completion, fusion, reasoning and AI applications	Comparison of embedding models on FB15K, WN11, WN18, WN18RR	KGs help recommender, QA and retrieval systems; multi-hop reasoning on large KGs is computationally expensive

ID	Citation / DOI	Problem addressed	Method / architecture	Dataset / evaluation	Key findings
P8	Sahlab, Kahoul, Jazdi & Weyrich (2022), Procedia Computer Science 207, 2814-2822. doi:10.1016/j.procs.2022.09.339	Screening and assessing growing numbers of publications in systematic literature reviews (SLRs)	Interfaces to digital libraries, embedded search criteria, NLP filtering and classification, results stored as a KG	Case study: SLR on context-awareness in automation systems, compared with a manual review	Partly automated SLR gave results considerably close to the manual review; quality assessment remains manual
P9	Peng, Zhu, Liu, Bo, Shi, Hong, Zhang & Tang (2025), ACM Trans. Information Systems 44(2), 1-52. doi:10.1145/3777378	Plain text RAG misses relations between entities	Survey organising GraphRAG into graph-based indexing, graph-guided retrieval and graph-enhanced generation	Survey of GraphRAG methods, tasks and evaluation benchmarks	Graph structure improves retrieval of relational knowledge; evaluation and scalability remain open
P10	Lin (2025), Proc. ACDSA 2025, 1-5. doi:10.1109/ACDSA65407.2025.11165842	Reasoning over long-tail relations with very few examples	Hierarchical (entity, triplet, context) few-shot KG completion with meta-learning, transfer learning and adversarial training	Academic knowledge graph data with long-tail relations	Reports better few-shot completion than the compared methods, at higher model complexity
P11	Choi & Jung (2025), Applied Sciences 15(7), 3727. doi:10.3390/app15073727	No common view of how KGs should be built and evaluated	Review of extraction (incl. multimodal), learning paradigms (TransE, GNNs, Transformers, hybrid RL) and evaluation	Reviews datasets across domains (general, healthcare, legal, temporal, multilingual)	LLM-driven extraction produces hallucinated triples unless post-processed; evaluation needs multidimensional metrics
P12	Zheng, Dong, Wang, Liu, Wang & Li (2025), arXiv:2501.02711	LLM-based KG completion focuses on triplet classification, not ranking	KG-CF: LLM labels reasoning paths, a distilled sequence classifier filters context, a BERT scorer ranks candidates	NELL-995, FB15K-237, WN18RR; Hits@1, MRR	Filtering irrelevant graph context improves ranking-based completion
P13	Chen & Shiu (2025), arXiv:2505.07618	Generating exam questions with controlled difficulty	KAQG: multi-agent RAG over per-subject KGs, Item Response Theory and Bloom's taxonomy	Self-constructed question set and human evaluation	Per-subject graphs prevent cross-domain contamination; psychometric weights need larger-scale calibration
P14	Dang, Tang, Pang, Wang, Li & Li (2021), J. Computer Science and Technology 36(5), 1200-1211. doi:10.1007/s11390-020-0328-2	Unifying MOOC course knowledge from several platforms	Ontology-based educational KG; concepts extracted from course text and linked to Wikipedia; prerequisite relations mined	Coursera, edX, XuetangX, ICourse: 52,779 entities, over 300,000 triples	Linking to Wikipedia normalises concepts; extracted prerequisite relations improve graph connectivity

ID	Citation / DOI	Problem addressed	Method / architecture	Dataset / evaluation	Key findings
P15	Wang, Zhao, Tan, Xiao & Cheng (2025), Science China Information Sciences 68(10), 202101. doi:10.1007/s11432-024-4329-3	Temporal KG models cannot reason about entities never seen in training	DM-UEP: diffusion model generates representations for unseen entities; two-phase reasoning (graph extension + Transformer)	Three open-world test sets built from public temporal KGs; MRR, Hits@K	Improves unseen-entity prediction over baselines; diffusion adds inference latency

2.1 Comparative Synthesis

Approaches. The papers fall into three groups. KG construction work (P1, P2, P5, P11, P14) builds graphs from structured data or text, mostly with general-purpose extractors or manual ontologies. KG reasoning and completion work (P7, P10, P12, P15) improves inference over an existing graph. KG-grounded generation work (P3, P4, P9, P13) uses graphs to reduce LLM hallucination. Only P8 targets scientific literature itself, and it stops at a browsable graph.

Architecture and cloud platform. None of the papers describes a deployed cloud architecture with authentication, event-driven ingestion and monitoring. P6 is distributed but simulated; the others are offline experiments or single-machine prototypes. Security and access control are not discussed in any of them.

Scalability, performance and cost. P7 and P10 note the computational cost of multi-hop reasoning and complex models, and P15 reports slower inference for diffusion. No paper reports end-to-end latency, throughput or operating cost of a running service.

Limitations shared across the work. Extraction quality on domain terms (P5, P11, P14), dependence on fragile inputs (P8), self-built evaluation sets without a controlled baseline (P3, P13), and lack of deployment evidence (all). These observations shape the objectives in Section 3.

2.2 Individual Research Gap Analysis

Each team member analysed five papers related to their area of the project: Devesh Agarwal (data and NLP extraction), Dhruv Gupta (knowledge graph and GraphRAG) and Parth Sharma (cloud deployment and application). Table 2 records each analysis and the improvement the project makes in response.

Table 2. Individual research gap analysis

Paper	Analyst	Existing method	Advantages	Limitations	Research gap identified	Possible improvement
P2	Devesh	Deep-learning pipelines for extraction, fusion and reasoning	End-to-end learning from unstructured text	Treated as a review: no deployed system, cost or latency figures	Construction methods are rarely evaluated as a running service with throughput and failure rates	Serve the extraction model behind an event-driven pipeline and report documents/hour and failures (done: Section 10)
P5	Devesh	LLM/spaCy extraction plus clustering and link prediction	Connects fragmented subgraphs; human-readable clusters	General-purpose extractors are noisy on scientific terms; embedding models need retraining for new entities	Domain-specific entity types (Method, Task, Dataset, Metric) need a model trained on scientific text	Fine-tune SciBERT on SciERC and canonicalise names before writing the graph
P8	Devesh	Library interfaces, NLP screening and a KG of SLR results	Automates acquisition and screening	Depends on publisher web interfaces; no full-text reading or natural-language querying	The resulting graph is browsed, not asked; publisher scraping is fragile	Use open APIs (OpenAlex, S2ORC) instead of scraping and add GraphRAG question answering
P11	Devesh	Survey of extraction, learning and evaluation	Broad, recent overview across domains	Notes that LLM extraction hallucinates triples without post-processing	Need for measurable extraction quality and graph-level effects, not only model scores	Report entity- and relation-level F1 on a public test set and the before/after effect on the graph
P14	Devesh	Ontology-driven KG with concepts linked to Wikipedia	Large unified KG; good linking accuracy for common concepts	Niche concepts missing from Wikipedia link poorly; relies on explicit text	Scientific methods and datasets are often too new or niche for encyclopaedia linking	Canonicalise with learned abbreviations and alias tables, and enrich papers from OpenAlex metadata
P3	Dhruv	Zero-shot LLM triplet extraction and two-stage KG-RAG	High accuracy without fine-tuning the LLM	Accuracy depends on KG coverage; single domain; self-built test set	No measurement of the same LLM with and without the graph on the same questions	Run a controlled benchmark: same model, same questions, with and without graph context (done: 0% vs 62.5% hallucination)
P4	Dhruv	Roadmap for unifying LLMs and KGs	Clear taxonomy of integration patterns	Conceptual; few deployable reference systems	Few end-to-end systems show KG-grounded LLM answers with citations at low cost	Build a deployed, cited GraphRAG system on serverless cloud services
P9	Dhruv	Survey of GraphRAG indexing, retrieval and generation	Systematic view of the design space	Limited guidance on latency and cost of graph retrieval	Multi-hop retrieval latency inside API time limits is rarely reported	Run graph queries in parallel and time-box the LLM (measured 0.535 s retrieval; 30 s API limit never exceeded)
P12	Dhruv	LLM-guided context filtering for KG completion	Removes redundant paths before scoring	Simple path context only; less stable in inductive settings	Retrieved context must be ranked by relevance to the question, not only filtered	Score facts by match quality, relation type and question wording before passing the top 30 to the LLM
P13	Dhruv	Multi-agent RAG over per-subject KGs	Prevents cross-domain interference	Small human evaluation; weights not calibrated	Grounded generation needs an objective, automatic evaluation	Generate questions with ground truth from the graph and score recall, hallucination and citation precision automatically
P1	Parth	Human-centric KG with ontologies and semantic queries	Captures multi-hop operational relations	Manual ontology engineering; structured inputs only	No automated pipeline from unstructured text to the graph	Event-driven ingestion: S3 upload triggers Lambda, SageMaker extraction and graph writes
P6	Parth	Decentralised knowledge sharing with GraphSAGE	No central bottleneck	Evaluated in simulation only	Cloud deployment, security and monitoring of KG services are not addressed	Deploy as managed serverless services with Cognito, IAM least privilege and CloudWatch monitoring

Paper	Analyst	Existing method	Advantages	Limitations	Research gap identified	Possible improvement
P7	Parth	Survey of KG technologies and applications	Comprehensive; identifies multi-hop cost	Survey only	Practical bounds on multi-hop cost in an interactive application are missing	Limit expansion to 3 hops and 27 bounded parallel queries; measure latency
P10	Parth	Hierarchical few-shot KG completion	Handles long-tail relations	High model complexity; no deployment discussion	Heavy models are costly to serve; inference cost matters for cloud systems	Serve compact BERT-size models on a serverless endpoint that costs nothing when idle
P15	Parth	Diffusion model for unseen entities	Works without auxiliary information	Slower inference due to diffusion	Graphs of literature grow continuously; new entities must appear without retraining	Add new papers incrementally with idempotent MERGE writes so new entities are queryable immediately

2.3 Consolidated Research Gap

The reviewed work shows that knowledge graphs reduce LLM hallucination (P3, P4, P9) and capture multi-hop relations (P1, P7), and that deep models can extract entities and relations from text (P2, P11). However, no reviewed system combines (a) automatic extraction of scientific entities and typed relations from literature, (b) a continuously updated graph, (c) question answering that cites its sources and is measured against the same LLM without the graph, and (d) a secure, monitored, low-cost cloud deployment. This project addresses that combined gap.

3. Project Objectives

1. **Automated extraction.** Extract Method, Task, Dataset and Metric entities and typed relations from scientific text with fine-tuned models. *Success criterion:* entity F1 of at least 0.60 and relation F1 of at least 0.60 on the SciERC test set, and every ingested paper linked to at least one entity.
2. **Serverless processing.** Process uploaded and fetched papers automatically with an event-driven AWS Lambda pipeline. *Success criterion:* at least 1,000 documents per hour with parallel functions and zero lost documents.
3. **Grounded answers.** Answer natural-language questions with GraphRAG. *Success criterion:* hallucination rate below 5% on a benchmark with ground truth, lower than the same LLM without the graph, and every answer citing source papers.
4. **Graph storage.** Store entities and relationships in a managed graph database reachable from the cloud functions. *Success criterion:* graph retrieval under 1 s on average and a graph of at least 20,000 nodes.

5. **Interactive dashboard.** Visualise the graph and multi-hop connections in a secure web dashboard. *Success criterion:* Ask, Explore, Connect, Research 360 and Upload work for signed-in users, and unauthenticated API calls are rejected.
6. **Operability.** Monitor, alert and test the deployed system. *Success criterion:* CloudWatch dashboard and alarms in place, an automated end-to-end test passing, and no card charges on the AWS Free plan.

3.1 Expected Outcomes

The deliverables are: the deployed system (dashboard URL and API); the Git repository with infrastructure as code (AWS SAM), source code, trained-model scripts and tests; the evaluation reports in results/; and this report. Objective 1 is assessed on the public SciERC test split; Objective 2 from CloudWatch and ingestion logs; Objective 3 with an automatic benchmark (scripts/evaluate_graphrag.py); Objective 4 from graph statistics and retrieval timings; Objectives 5 and 6 with the end-to-end test (scripts/smoke_test.py) and the CloudWatch console. Results are mapped to objectives in Section 10.4.

4. Novelty Summary

Table 3. Novelty compared with existing work

Dimension	Existing approach / limitation	Proposed difference	Evidence / evaluation
New feature	Literature KGs are browsed (P8); answers are not cited	Plain-English questions answered only from graph facts with clickable paper citations; multi-hop Connect; Research 360 profiles	100% of answers cite sources; 98.3% citation precision (Section 10.2)
Algorithm / processing	General-purpose or rule-based extractors (P5); LLM extraction hallucinates triples (P11)	Two SciBERT models (NER; RE with typed entity markers and entity-start pooling) plus canonicalisation	NER F1 0.618; RE F1 0.661 vs 0.423 for the earlier RE model; papers with entities 189 to 305 of 305
Cloud architecture / integration	Offline prototypes or simulations (P1, P6)	Event-driven serverless pipeline: S3, Lambda, SageMaker serverless, API Gateway, Cognito, Amplify, CloudWatch; graph and LLM backends switchable by parameter	306 documents in 13.5 min, 0 failures; 13/13 end-to-end checks; 0 server errors in 240 API requests
Security / automation / scalability	Not addressed in the reviewed papers	Cognito JWT on every private route, least-privilege IAM per function, encrypted private S3, dead-letter queue, alarms, retries with back-off	Unauthenticated request rejected (401); throttling-safe ingestion with 12 parallel functions

Central novelty claim. The project combines automatic SciBERT-based extraction of scientific entities and typed relations with a GraphRAG question-answering layer, and deploys

both as a secure, event-driven serverless system on AWS. Unlike the reviewed KG-RAG work, grounding is measured against the same LLM without the graph on identical questions: the hallucination rate falls from 62.5% to 0%. The system also shows that this can run at near-zero cost by keeping every compute component serverless and making the graph and LLM providers deployment parameters. We do not claim that any single component is new; the contribution is the integrated, measured, deployable system.

5. Proposed System and Architecture

5.1 System Overview

Users. Researchers sign up with e-mail verification and can ask questions and explore the graph. Project members (a Cognito group) can also upload papers. **Inputs.** OpenAlex metadata and abstracts, S2ORC full text, open-access arXiv full text, and PDF or JSON uploads. **Processing.** An ingest Lambda function parses each document, repairs PDF text, resolves S2ORC inline citations, matches the paper to OpenAlex, calls the SciBERT endpoint for entities and relations, canonicalises names and writes the graph. **Outputs.** Cited answers, an interactive graph view, multi-hop paths between any two entities, entity profiles and live upload status.

5.2 Diagram 1 - Cloud Deployment Architecture

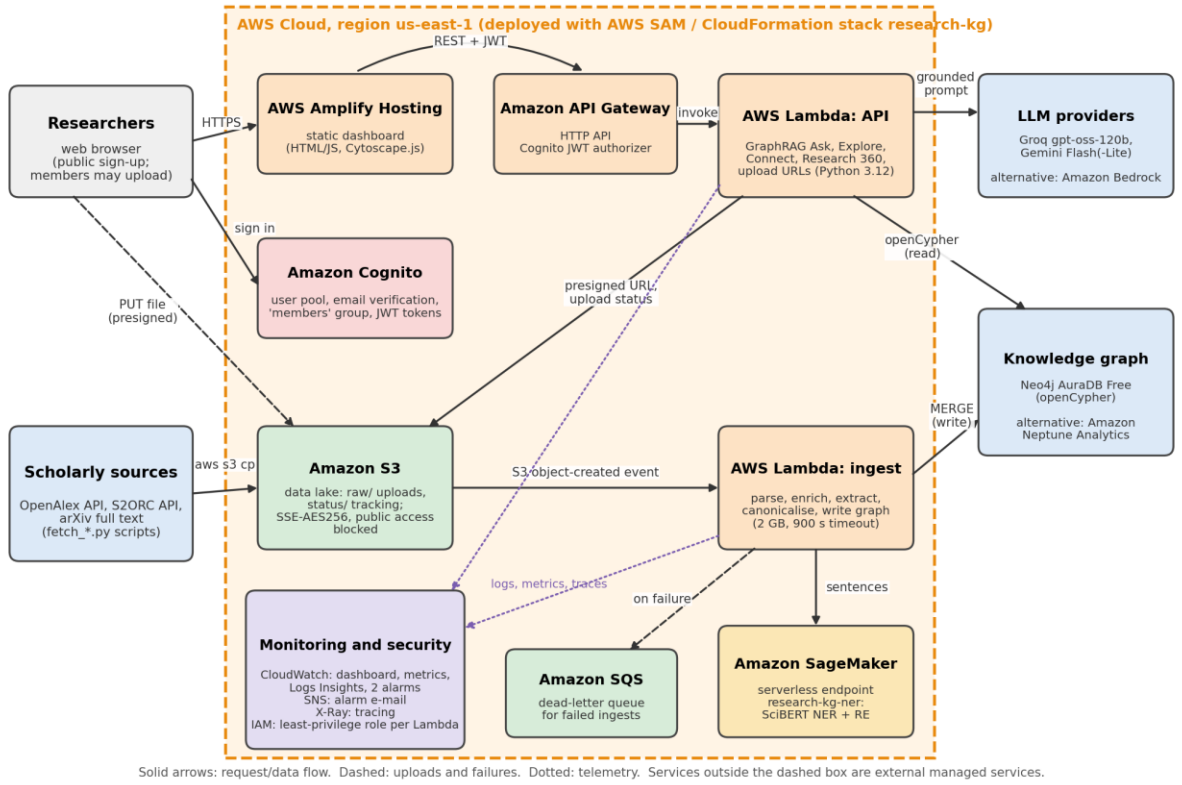


Figure 1. *Cloud Deployment Architecture (as deployed in AWS us-east-1).*

Components and data flow. The dashboard is a static site on **AWS Amplify Hosting**. Users authenticate against an **Amazon Cognito** user pool directly from the page and receive JSON Web Tokens. Each API call goes to an **Amazon API Gateway** HTTP API whose JWT authorizer validates the token before invoking the **API Lambda** function. That function runs GraphRAG retrieval against the knowledge graph (openCypher over TLS to **Neo4j AuraDB Free**, or **Amazon Neptune Analytics** when GraphBackend=neptune-analytics) and sends only the retrieved facts to the LLM (**Groq** gpt-oss-120b with **Google Gemini** fallback, or **Amazon Bedrock** when selected). For uploads it returns a presigned **Amazon S3** URL, and the browser uploads the file straight to the raw/ prefix. Fetch scripts copy OpenAlex and S2ORC records to the same prefix. Each new object raises an S3 event that invokes the **ingest Lambda** function, which calls the **Amazon SageMaker** serverless endpoint (SciBERT NER + RE) and MERGEs the result into the graph. Failed ingest events go to an **Amazon SQS** dead-letter queue. Both functions publish logs, custom metrics and **AWS X-Ray** traces to **Amazon CloudWatch**; two alarms notify the team through **Amazon SNS** e-mail. Each function has its own **AWS IAM** role. The whole stack is defined in one AWS SAM template (template.yaml).

5.3 Diagram 2 - Complete System Architecture / Workflow

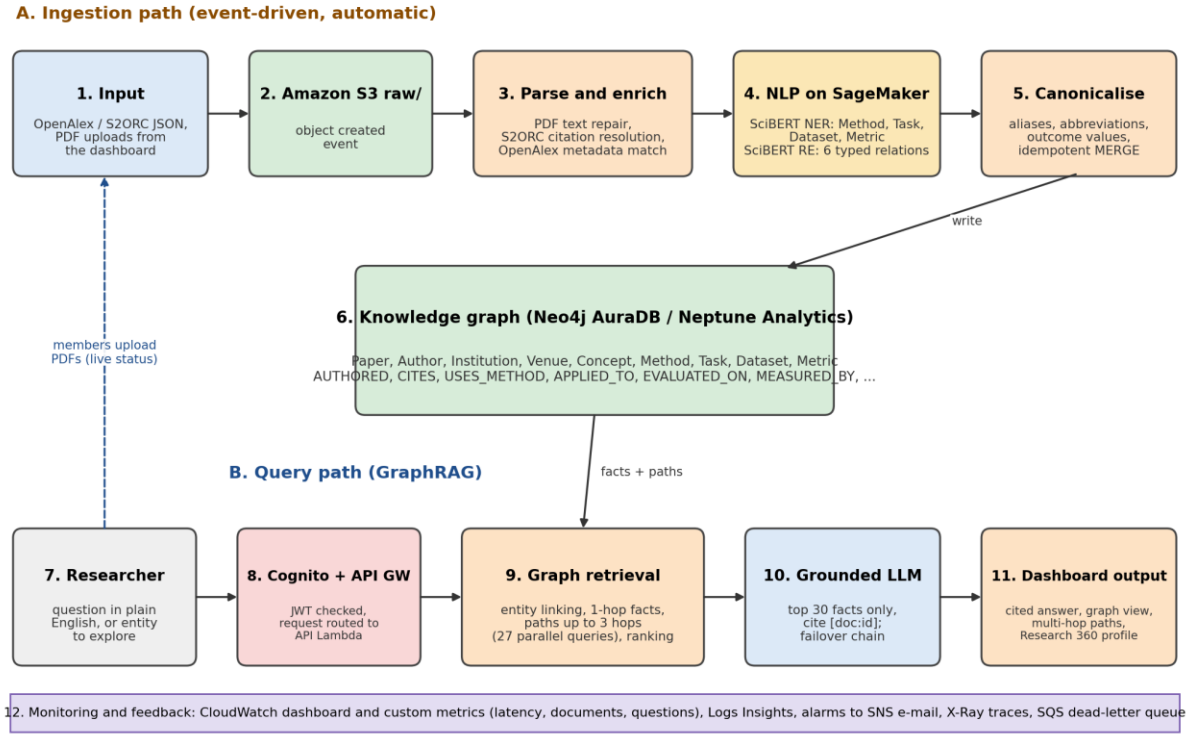


Figure 2. *Complete System Architecture and Workflow (ingestion path A and query path B).*

From input to output. In path A, a document enters S3 (steps 1-2), is parsed and enriched (3), passes through the two SciBERT models on SageMaker (4), and its entities, relations and reported outcome values (for example F1 = 91.4%) are canonicalised and merged into the graph (5-6). In path B, a researcher's question passes Cognito and API Gateway (7-8); the API function links question phrases to graph nodes, collects their neighbours and paths of up to three hops, and ranks the facts (9); the top 30 facts go to the LLM with an instruction to answer only from them and cite [doc:id] (10); and the dashboard renders the cited answer, the graph and the paths (11). Monitoring (12) covers both paths. Members' uploads feed back into path A and their progress is shown live.

5.4 Workflow and Use Cases

Use case 1, ask a question.

1. The researcher signs in; the page stores the Cognito tokens and refreshes them automatically.
2. The researcher types "Which methods are used for link prediction?" and presses Ask.
3. The API function links "link prediction" to Task nodes, runs 27 bounded graph queries in parallel and ranks the facts.

4. The LLM writes a short answer from the top 30 facts; each statement ends with a citation such as [W3120491054].
5. If no LLM answers within the 22-second budget, the ranked facts are returned instead, so the request never fails.

Use case 2, upload a paper (members). The member chooses a PDF; the page requests a presigned URL, uploads the file to S3 and polls the status object while the ingest function processes it; the new paper is then searchable.

Use case 3, explore and connect. The researcher enters an entity (for example "Graph Neural Network") to see its neighbourhood, or two entities (TransE and BERT) to see the shortest multi-hop paths between them; clicking a node opens its Research 360 profile.

5.5 Security, Availability and Scalability

- **Authentication and access control:** every API route except the public statistics route requires a valid Cognito JWT; uploads additionally require membership of the members group, checked in the function. Passwords need at least 10 characters with upper- and lower-case letters and digits, and e-mail addresses are verified.
- **Least privilege:** the ingest role can read the bucket, write status objects and invoke only the research-kg-ner endpoint; the API role can write only raw/uploads/* and read status/*. No function has administrator rights.
- **Encryption and storage:** the S3 bucket uses server-side encryption (SSE-S3, AES-256) and blocks all public access; uploads use short-lived presigned URLs. Graph and LLM traffic uses TLS. Lambda environment variables are encrypted at rest by AWS.
- **Secrets:** database passwords and API keys are NoEcho CloudFormation parameters, never committed to Git (.gitignore covers credential files).
- **Fault handling:** the ingest function retries once and then sends the event to the SQS dead-letter queue (14-day retention); SageMaker throttling is handled with exponential back-off and jitter; LLM calls fail over across providers and finally to cited facts.
- **Backups and availability:** source data stays in S3 and in the public APIs, so the graph can be rebuilt from raw/ by re-running ingestion (measured at 13.5 minutes for 306 documents). Lambda, API Gateway, S3 and SageMaker serverless are multi-AZ managed services.

- **Scaling:** Lambda scales per event (12 parallel ingest functions were used), the SageMaker endpoint scales to the account maximum of 10 concurrent invocations and to zero when idle, and the graph backend can be resized (GraphMemory parameter for Neptune Analytics).

6. Dataset / Input Data Description

Three data sources are used: OpenAlex and S2ORC as inputs to the knowledge graph, and SciERC to train and test the NLP models. Raw data is fetched by scripts and stored in S3, not in Git; dataset/sample_papers.jsonl holds four sample papers in the project's document format.

Table 4. Dataset 1: OpenAlex

Attribute	Details
Dataset / input name	OpenAlex scholarly works (with authors, institutions, venues, concepts, references)
Source / organisation	OurResearch (non-profit)
URL / access method	REST API https://api.openalex.org (scripts/fetch_openalex.py); full snapshot also on S3 (s3://openalex)
Size	Full snapshot about 330 GB compressed; project subset: about 300 works per search, plus about 20,000 referenced works as Paper nodes
Number of records	Over 300 million works in the full index; 306 documents fully ingested in the final graph
Number of features	About 50 top-level fields per work; the project uses id, DOI, title, abstract (inverted index), year, venue, authorships with institutions, concepts, referenced works, retraction flag
Data type / format	JSON (API responses), stored as JSON Lines in S3
License / restrictions	CC0 (public domain)
Purpose	Seed papers with exact metadata and citation edges; enrich uploaded PDFs
Preprocessing	Rebuild abstracts from the inverted index; drop retracted works (is_retracted filter); normalise author and institution names; convert to the canonical document schema
Privacy / ethics	Public bibliographic metadata only; author names are public research records; no personal contact data stored

Table 5. Dataset 2: Semantic Scholar Open Research Corpus (S2ORC)

Attribute	Details
Dataset / input name	S2ORC full text via the Semantic Scholar API
Source / organisation	Allen Institute for AI (Lo et al., 2020)
URL / access method	Semantic Scholar Datasets API with a free API key (scripts/fetch_s2orc.py, scripts/prepare_s2orc.py)
Size	Full corpus hundreds of GB; project uses selected papers only
Number of records	Over 12 million full-text papers in the corpus; a small subset used for full-text testing

Attribute	Details
Number of features	Metadata, abstract, body paragraphs, section headings, cite_spans, ref_spans, bib_entries
Data type / format	JSON / JSON Lines
License / restrictions	ODC-BY for metadata; full text under the original open-access licences
Purpose	Full-text extraction of methods, results and citation context
Preprocessing	Join metadata and full text by paper ID; resolve inline citation markers (cite_spans ref_id such as BIBREF3) to bib_entries so a statement can be linked to the cited paper
Privacy / ethics	Published research text only

Table 6. Dataset 3: SciERC (model training and testing)

Attribute	Details
Dataset / input name	SciERC (Luan et al., 2018)
Source / organisation	University of Washington / AI2
URL / access method	Public download; prepared by src/ai_model/train/train.py and train_re.py
Size	About 4 MB
Number of records	500 abstracts; 1,861 train / 275 dev / 551 test sentences; 3,215 training relations
Number of features	6 entity types (Method, Task, Metric, Material, OtherScientificTerm, Generic); 7 relation types
Data type / format	JSON Lines with token-level span annotations
License / restrictions	Research use
Purpose	Fine-tuning and evaluating the SciBERT NER and RE models
Preprocessing	BIO tagging for NER; candidate entity pairs with typed markers for RE; one sampled NONE pair per positive for training
Privacy / ethics	Public scientific abstracts

6.1 Data Preparation

Validation: every document is converted to one canonical schema (src/backend/kg/schema.py) and rejected if it has no title or text. **PDF repair:** ligatures, hyphenation across line breaks and split small capitals are fixed before extraction. **Canonicalisation:** entity names are normalised with an alias table, learned abbreviations and plural removal; generic phrases ("dataset", "results") are dropped; each name gets one type by majority vote. **Idempotence:** graph writes use MERGE on stable IDs, so re-ingesting a paper never duplicates nodes. **Splits and leakage:** the models were trained on the SciERC train split; thresholds were tuned only on the dev split; all reported scores are on the untouched test split. The GraphRAG benchmark questions are generated from the graph itself with known answers, and the LLM-only baseline receives the identical questions.

7. Cloud Platform and Services Planning

The platform is AWS (region us-east-1), on the AWS Free plan. Status values: **Deployed** = running in the live stack; **Configured** = implemented and deployable by parameter, not running.

Table 7. Cloud services, purpose, status and considerations

Platform / service	Purpose	Integration / data flow	Status	Security / cost considerations
Amazon S3	Data lake for raw documents and upload status	Fetch scripts and presigned uploads in; S3 event to ingest Lambda	Deployed	SSE-S3, public access blocked, multipart clean-up after 7 days; free tier
AWS Lambda (ingest)	Parse, enrich, extract, write graph	S3 event in; SageMaker and graph out; DLQ on failure	Deployed	Own IAM role; 2 GB, 900 s; free tier at project scale
AWS Lambda (API)	GraphRAG, Explore, Connect, Insights, uploads	API Gateway in; graph, LLM and S3 out	Deployed	Own IAM role; 29 s timeout under API Gateway's 30 s
Amazon SageMaker	Serverless endpoint serving SciBERT NER and RE	Invoked by ingest Lambda	Deployed	3 GB memory, max concurrency 10; billed per request, $0 when idle
Amazon API Gateway	HTTP API for the dashboard	Dashboard in; API Lambda out	Deployed	Cognito JWT authorizer on all private routes; free tier
Amazon Cognito	Sign-up, sign-in, members group	Tokens used by dashboard and API Gateway	Deployed	E-mail verification, password policy; MFA not enabled (future work)
AWS Amplify Hosting	Hosts the dashboard	Static HTML/JS served over HTTPS	Deployed	No server to patch; free tier
Amazon CloudWatch, SNS, AWS X-Ray	Dashboard, custom metrics, Logs Insights, alarms, tracing	Logs and metrics from both functions and the endpoint	Deployed	Alarms on ingest and API errors e-mailed via SNS; free tier
Amazon SQS	Dead-letter queue for failed ingests	Ingest Lambda on-failure destination	Deployed	14-day retention; free tier
AWS IAM	Least-privilege roles per function	Attached by the SAM template	Deployed	Scoped to one bucket prefix and one endpoint
Neo4j AuraDB Free (external)	Knowledge graph database	openCypher over TLS from both functions	Deployed	Free tier: 200k nodes / 400k relationships; pauses after 3 idle days
Groq, Google Gemini (external)	LLM for grounded answers	Called by API Lambda with retrieved facts only	Deployed	Free tiers; keys as NoEcho parameters
Amazon Neptune Analytics	Managed graph (original design)	Same openCypher queries via IAM-signed requests	Configured	Blocked on Free plan; about $0.48/hour at 16 m-NCU when enabled
Amazon Bedrock	Managed LLM (Nova / Claude)	Selected by LlmProvider parameter	Configured	Blocked on Free plan; pay per token

7.1 Deployment and Network Design

All resources are in us-east-1 and are created from one AWS SAM template (template.yaml) with sam build and sam deploy; the stack is named research-kg and the stage dev. No VPC is used: every service is a managed regional endpoint, which avoids NAT gateway charges, and each call is authenticated (IAM SigV4 between AWS services, Cognito JWT for users, TLS with credentials for Neo4j and the LLM APIs). Configuration such as GraphBackend,

LlmProvider, ExtractorMode and model IDs are template parameters, so one template serves both the free deployment and the full AWS design. The dashboard is deployed to Amplify from src/frontend with a generated config.js holding only public identifiers (API URL, user pool and client IDs).

7.2 Cost Estimation

Actual expenditure (AWS Free plan, October 2026): $0 charged to the card. Lambda, API Gateway, S3, Cognito, CloudWatch, SQS and Amplify stayed within free-tier limits; SageMaker serverless inference used a few cents of the plan's credits across all ingestion runs; Neo4j AuraDB Free, Groq and Gemini free tiers cost $0.

Estimate for the full AWS design (assumptions: Neptune Analytics graph at 16 m-NCU running only during an 8-hour working day on 22 days, about 1,000 questions and 2,000 ingested papers per month; prices from the AWS pricing pages for us-east-1 as noted in docs/AWS_IMPLEMENTATION.md, to be rechecked before use):

Table 8. Monthly cost estimate for the full AWS design (estimate, not actual spend)

Item	Assumption	Estimated monthly cost (USD)
Neptune Analytics, 16 m-NCU	176 hours x about $0.48/hour	about 85
SageMaker serverless inference	2,000 papers x about 7 invocations, 3 GB, about 3.5 s each	about 3
Amazon Bedrock (Nova Lite)	1,000 questions x about 4,000 input tokens	under 1
Lambda, API Gateway, S3, Cognito, CloudWatch, SQS, Amplify	Within free-tier limits at this volume	about 0-2
Total		about 90 (about 350 if the Neptune graph runs 24/7)

Cost controls: an AWS Budgets alert is recommended at $25 with an e-mail at 80% (docs/AWS_IMPLEMENTATION.md); the Neptune graph exists only while GraphBackend=neptune-analytics and is removed by redeploying with neo4j; the SageMaker endpoint is serverless and costs nothing when idle.

7.3 Portability / Provider-Specific Choices

Provider-specific parts are Lambda handlers, the SAM template, Cognito tokens, the SageMaker inference container and S3 events. Portable parts are the extraction and GraphRAG library (src/backend/kg), which runs locally (scripts/local_ingest.py, scripts/ask.py), the openCypher queries (identical on Neo4j and Neptune), and the models (standard Hugging Face format). The graph and LLM layers are already behind interfaces (graph_client.py, llm.py),

which is how the project moved from Neptune and Bedrock to Neo4j and Groq/Gemini without code changes. Moving to another cloud would mainly require rewriting the template and the two handler entry points.

8. Methodology and Implementation Plan

8.1 Overall Methodology

The project followed an incremental process: requirements and literature review (proposal); data preparation (fetchers, parsers, schema); design (ontology, architecture, SAM template); implementation in feature branches (ingestion and NLP, knowledge graph and GraphRAG, dashboard and infrastructure); integration on AWS; and evaluation with reproducible scripts. A proposal review then compared the system with every promise in the proposal and added relation extraction, outcomes, Connect, Research 360 and retraction filtering.

8.2 Tools and Technologies

Table 9. Tools and technologies

Tool / technology	Purpose	Version / configuration
AWS (S3, Lambda, SageMaker, API Gateway, Cognito, Amplify, CloudWatch, X-Ray, SNS, SQS, IAM)	Cloud platform	us-east-1; stack research-kg, stage dev; AWS SAM CLI
Python	Lambda functions, pipeline, scripts, tests	Python 3.12 (Lambda runtime); pytest
PyTorch, Hugging Face Transformers	SciBERT fine-tuning and inference	allenai/scibert_scivocab_uncased; SageMaker Hugging Face container (transformers 4.49, PyTorch 2.6, CPU)
Neo4j AuraDB Free / Amazon Neptune Analytics	Knowledge graph	openCypher; Neo4j Python driver
Groq, Google Gemini, Amazon Bedrock	LLM layer	openai/gpt-oss-120b; gemini-flash-lite-latest; Nova 2 Lite (option)
HTML, CSS, JavaScript, Cytoscape.js	Dashboard and graph visualisation	Cytoscape.js 3.30.2
Git and GitHub	Version control and collaboration	Repository on GitHub; main branch plus feature branches; 5 pull requests

8.3 Step-by-Step Implementation Plan

1. **Requirements and success criteria:** objectives with measurable criteria (Section 3).
2. **Data preparation:** OpenAlex and S2ORC fetchers, PDF parser, canonical schema, retraction filter, SciERC preparation.

3. **Cloud resources and architecture:** SAM template with S3, two Lambda functions, API Gateway, Cognito, CloudWatch dashboard and alarms, SQS DLQ, optional Neptune Analytics.
4. **Application, backend and models:** SciBERT NER and RE training and calibration; SageMaker deployment; extraction and canonicalisation; graph writer; GraphRAG retrieval and LLM failover; dashboard.
5. **Integration and security:** S3 event wiring, JWT authorizer, least-privilege roles, presigned uploads, throttling back-off, members group.
6. **Testing and evaluation:** unit tests, end-to-end smoke test, NER/RE test scores, GraphRAG hallucination and retrieval benchmarks, CloudWatch metrics collection.
7. **Deployment, monitoring and documentation:** Amplify hosting, CloudWatch dashboard and saved queries, setup guides, evaluation reports and this report.

8.4 Evaluation Metrics

- **Extraction quality:** entity-level exact-match precision, recall and F1 (span and type) for NER; precision, recall and F1 over all candidate pairs of gold entities for RE (SciERC test).
- **Answer quality:** answer recall (share of ground-truth items in the answer), fully correct answers, hallucination rate (a confident answer that is mostly wrong), correct refusals for made-up entities, share of answers with citations and citation precision.
- **Retrieval:** context recall (ground-truth items present in the facts given to the LLM) and retrieval latency.
- **Performance and reliability:** seconds per document (median, p90), documents per hour, failures and dead-letter messages, API latency and 5xx errors, endpoint latency and cold starts (CloudWatch).
- **Cost:** actual charges on the AWS bill and estimated monthly cost for the full design.

9. Implementation / Development

All modules below are completed and deployed unless marked otherwise. Screenshots were taken from the live system on 15 October 2026 with a temporary test account that was deleted afterwards; no credentials are shown.

9.1 Implemented Modules

Table 10. Implemented modules

Module	Implementation details	Platform / services / tools	Evidence	Status
Data acquisition	OpenAlex and S2ORC fetchers, PDF upload, retraction filter	Python, S3	scripts/fetch_*.py; PR #1, PR #4	Completed
Parsing and enrichment	PDF repair, S2ORC citation resolution, OpenAlex matching	Lambda	src/backend/kg/parsers.py, enrich.py; PR #1	Completed
SciBERT NER	Fine-tuned on SciERC; served on SageMaker	PyTorch, SageMaker serverless	src/ai_model/train/train.py; commit b1c3d2d	Deployed
SciBERT RE	Typed markers, entity-start pooling, dev calibration	PyTorch, SageMaker	train_re.py, re_model.py; PR #4, PR #5	Deployed
Knowledge graph	Ontology, idempotent writer, Neo4j/Neptune clients	Neo4j AuraDB, Neptune Analytics	src/backend/kg/schema.py, graph_writer.py; PR #2	Deployed (Neo4j)
GraphRAG	Entity linking, parallel multi-hop retrieval, ranking, grounded LLM with failover	Lambda, Groq, Gemini, Bedrock	retrieval.py, llm.py; PR #2	Deployed
API and infrastructure	SAM template, API Lambda, Cognito, alarms, DLQ	SAM, API Gateway, Cognito, CloudWatch	template.yaml, handlers/api.py; PR #3	Deployed
Dashboard	Landing page, sign-in/up, Ask, Explore, Connect, Research 360, uploads	Amplify, Cytoscape.js	src/frontend/; PR #3, PR #4, PR #6	Deployed
Testing and evaluation	33 unit tests, end-to-end test, benchmarks, metrics, UI/UX and accessibility tests	pytest, boto3, Playwright, axe-core	tests/, scripts/smoke_test.py, results/; PR #6	Completed

9.2 Screenshots and Configuration Evidence

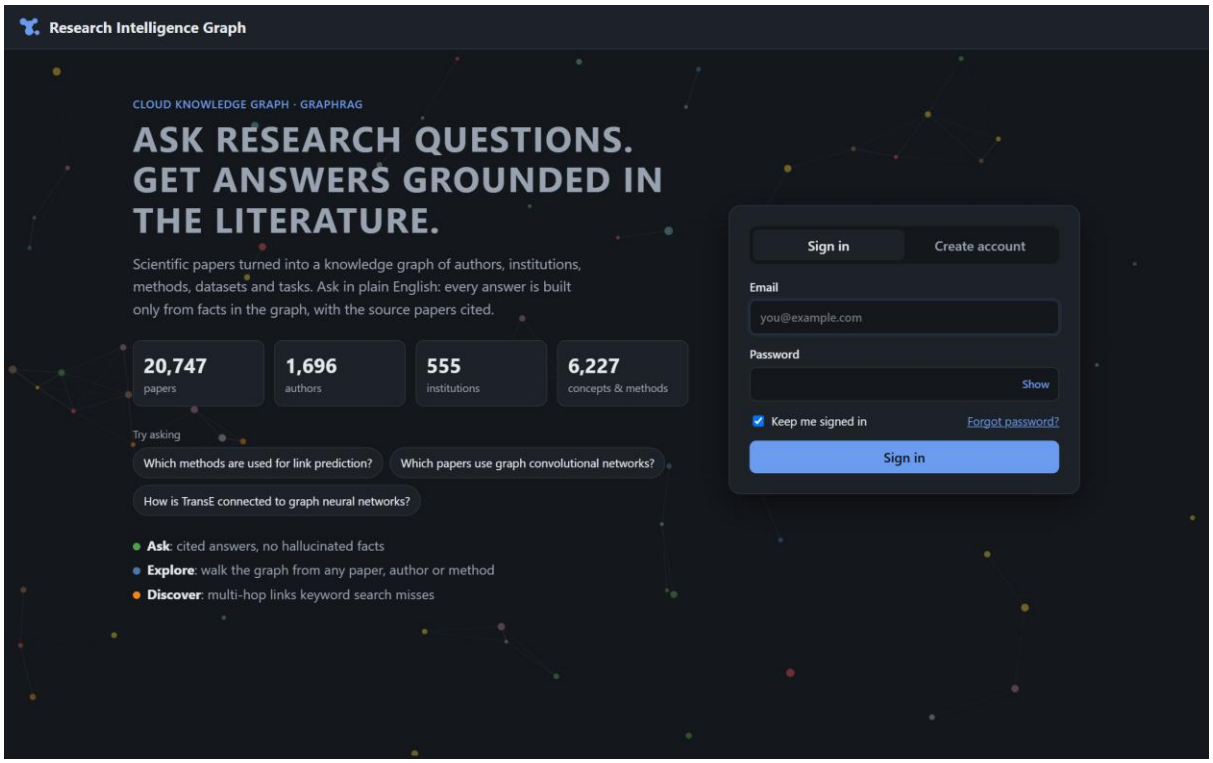


Figure 3. *Public landing page with live graph statistics and in-page Cognito sign-in (15 Oct 2026).*

The landing page reads /public/stats, the only route without authentication. Signing in uses the Cognito user-pool API directly from the page.



Figure 4. *Ask: GraphRAG answer to "Which methods are used for link prediction?" with citations, the graph of retrieved facts, and graph size (signed-in member).*

Each statement in the answer ends with a clickable citation to its source paper (OpenAlex ID or sample document). The graph in the centre shows the facts that were retrieved, and the right panel shows the live node counts and the members-only upload control.

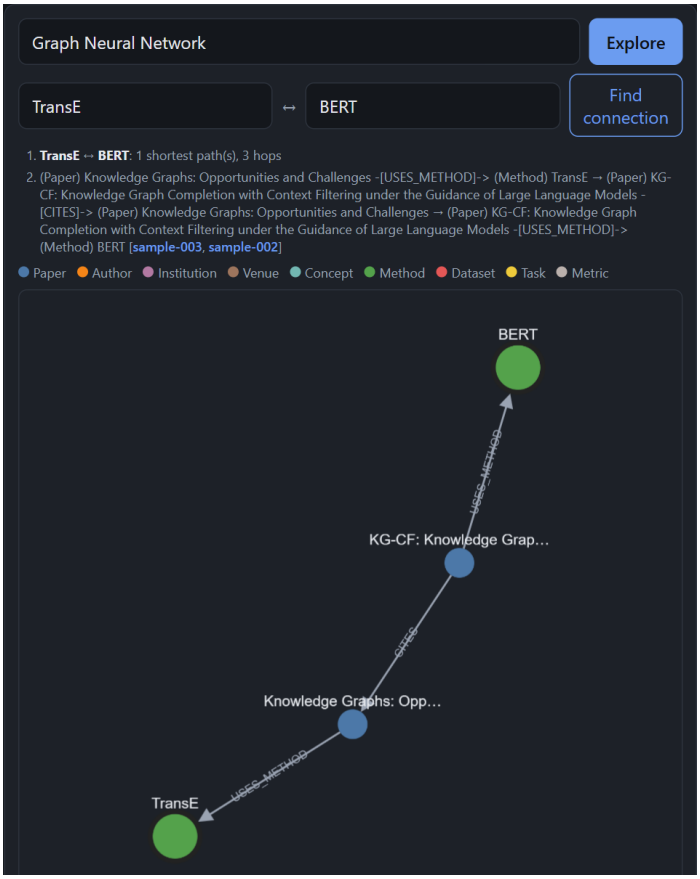


Figure 5. *Connect: the shortest multi-hop path between TransE and BERT (TransE is used by a survey that KG-CF cites, and KG-CF uses BERT).*

Configuration evidence in the repository: template.yaml (all AWS resources and parameters), src/aws/ (IAM policy documents for the SageMaker role), and results/system_metrics.json (CloudWatch metrics exported by scripts/collect_metrics.py). The CloudWatch dashboard research-kg-dev, the four saved Logs Insights queries and the two alarms are listed in results/EVALUATION.md, section 7.

9.3 Repository and Project Structure

Repository:
https://github.com/DeveshAg30/Cloud_Based_Knowledge_Graph_Framework_for_Automated_Research_Intelligence_Aws_Project_2026

Folder	Contents
src/backend/	Lambda code: kg/ (parsers, extractors, aliases, enrichment, schema, graph client and writer, retrieval, LLM, insights, metrics) and handlers/ (ingest, api)
src/ai_model/	SciBERT NER and RE training, calibration and SageMaker inference handler
src/frontend/	Dashboard (index.html, app.js, style.css, config.example.js)
src/aws/	IAM policy documents; the SAM template is template.yaml at the root

Folder	Contents
scripts/	Fetchers, local ingest, SageMaker launch, benchmarks, metrics collection, smoke test
dataset/	Four sample papers in the canonical format (raw data is fetched, not committed)
results/	Evaluation reports and raw metric files
docs/, architecture/	Setup guides (free tier and full AWS) and architecture notes
literature_survey/, references/, presentation/	Literature survey tables, reference list, UI/UX testing report and presentation material
tests/	Unit tests (pytest)

9.4 Version Control Workflow

The repository uses a protected main branch and one feature branch per work area: feature/data-ingestion-nlp, feature/knowledge-graph-graphrag, feature/dashboard-api-infra, fix/proposal-review and feature/re-improvement. Work was committed in small, descriptive commits and merged only through pull requests: #1-#5 were reviewed and merged by the repository owner, and #6 (this report, the testing documents and the fixes found during testing) was open for review at the time of writing. The branches were based on main in order, so no merge conflicts had to be resolved. Release tags have not been created yet; tagging the final commit as v1.0 is planned before submission.

10. Testing, Results and Observations

10.1 Test Plan and Test Cases

Test environment: the deployed stack in AWS us-east-1 (Neo4j AuraDB Free, Groq/Gemini), run from a Windows laptop with scripts/smoke_test.py on 15 October 2026 after the final backend deployment, plus 33 unit tests (pytest, all passing).

Table 11. End-to-end test cases (scripts/smoke_test.py, run 15 Oct 2026)

Test ID	Scenario	Input / setup	Expected result	Observed result	Pass/Fail
T01	Functional: sign-in	Temporary Cognito user, USER_PASSWORD_AUTH	ID token issued	Token issued	Pass
T02	Functional: public statistics	GET /public/stats without token	200 with counts	200; 20,747 papers; 5.75 s (cold start)	Pass
T03	Security: unauthenticated access	GET /stats without token	401 rejected	401 in 0.90 s	Pass
T04	Security: authenticated access	GET /stats with JWT	200, member flag	200; 29,374 nodes; 0.94 s	Pass
T05	Functional: GraphRAG answer	"Which methods are used for link prediction?"	Answer under 30 s with citations	Answer in 5.79 s, 3 citations, AI summary	Pass

Test ID	Scenario	Input / setup	Expected result	Observed result	Pass/Fail
T06	Hallucination: made-up entity	"Which papers evaluate the Zelkova-9 benchmark?"	No invented facts	"The provided facts do not contain any information about papers that..."	Pass
T07	Functional: Explore	GET /graph for Graph Neural Network	Neighbourhood graph	58 nodes in 1.67 s	Pass
T08	Functional: Connect	GET /connect TransE to BERT	Distinct multi-hop paths	1 distinct path, 3 hops, 1.86 s (before the de-duplication fix the same path was listed 5 times)	Pass
T09	Functional: Research 360	GET /insights for Graph Convolutional Network	Entity profile	Method profile, 6 sections, 1.90 s	Pass
T10	Security: member upload URL	POST /upload-url as member	Presigned URL	Issued in 1.05 s	Pass
T11	Integration: ingestion	PUT sample file, poll status	Processed into graph	Done in 49.0 s: SciBERT on SageMaker, 22 edges	Pass
T12	Performance: bulk ingestion	Rebuild of 306 documents, 12 parallel Lambdas	>= 1,000 docs/hour, no losses	13.5 min (about 1,350/hour), 0 failures, 0 DLQ messages	Pass
T13	Performance: retrieval latency	51 benchmark questions	< 1 s average	0.535 s average	Pass

10.2 Results

Extraction (Objective 1)

SciBERT NER was trained for 4 epochs (learning rate 3e-5, batch 16) on a laptop CPU in about 35 minutes because the account's SageMaker training quota is zero; RE was trained for 4 epochs in about 3 hours, keeping the best epoch by dev F1. Scores are on the SciERC test split (551 sentences; 5,062 candidate pairs for RE).

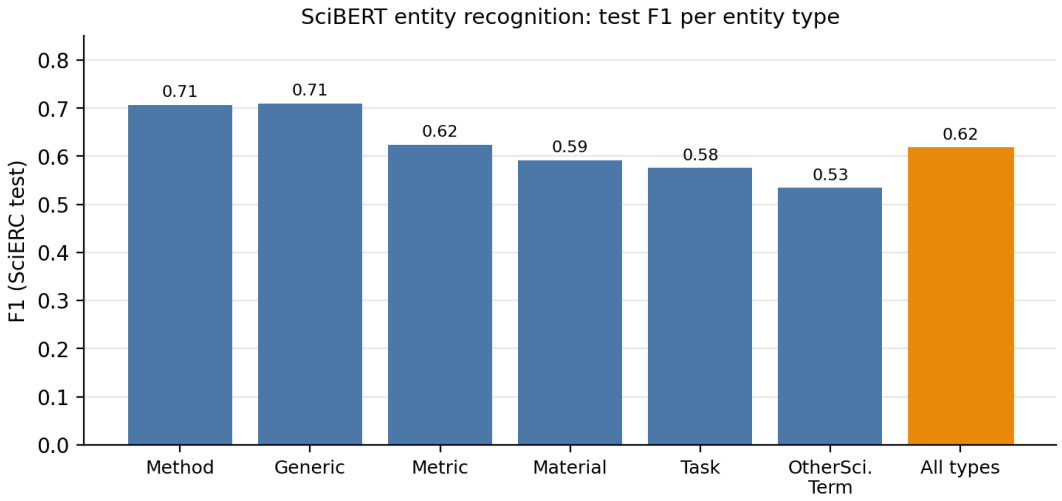


Figure 6. *SciBERT NER test F1 per entity type (overall 0.618; precision 0.593, recall 0.645).*

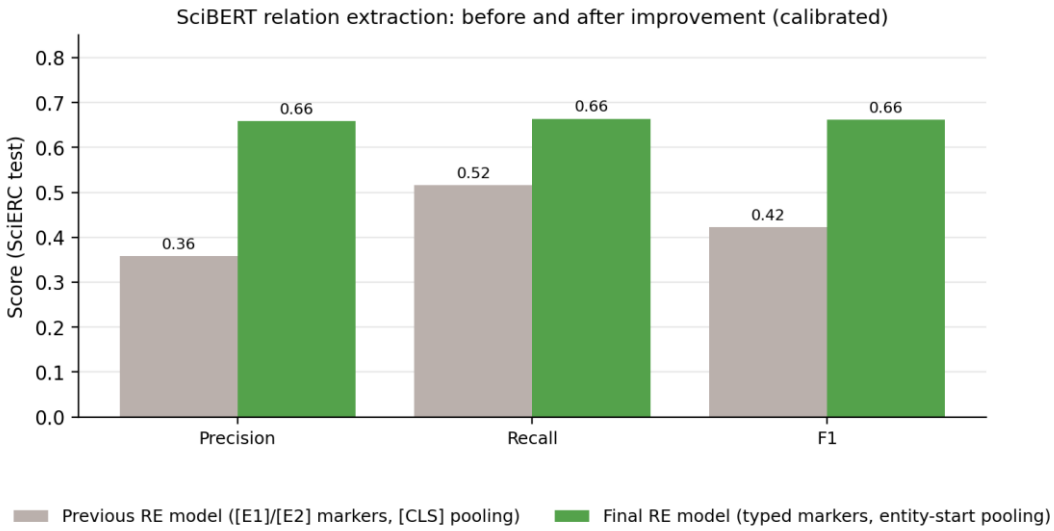


Figure 7. *Relation extraction before and after the model improvement (SciERC test, calibrated on dev).*

On the same corpus, the rule-based baseline linked entities to 189 of 304 papers; SciBERT plus the term dictionary linked entities to all 305, raised paper-to-entity links from 377 to 6,063 and created 2,630 method-to-task/dataset/metric relations. The RE model adds 865 typed relations (APPLIED_TO 399, IS_A 191, EVALUATED_ON 122, MEASURED_BY 83, COMPARED_WITH 51, PART_OF 19) and 64 reported outcome values.

Knowledge graph and retrieval (Objective 4)

Final graph (evaluation snapshot): 29,320 nodes and 42,015 relationships: 20,746 papers (most are cited references), 1,674 authors, 553 institutions, 2,416 methods, 1,727 tasks, 779 datasets and 262 metrics. On 51 generated questions, context recall was 98.3%, 97.8% of questions had every ground-truth item retrieved, and average retrieval time was 0.535 s with 27 parallel queries (previously about 6 s with 44 sequential queries).

GraphRAG answers (Objective 3)

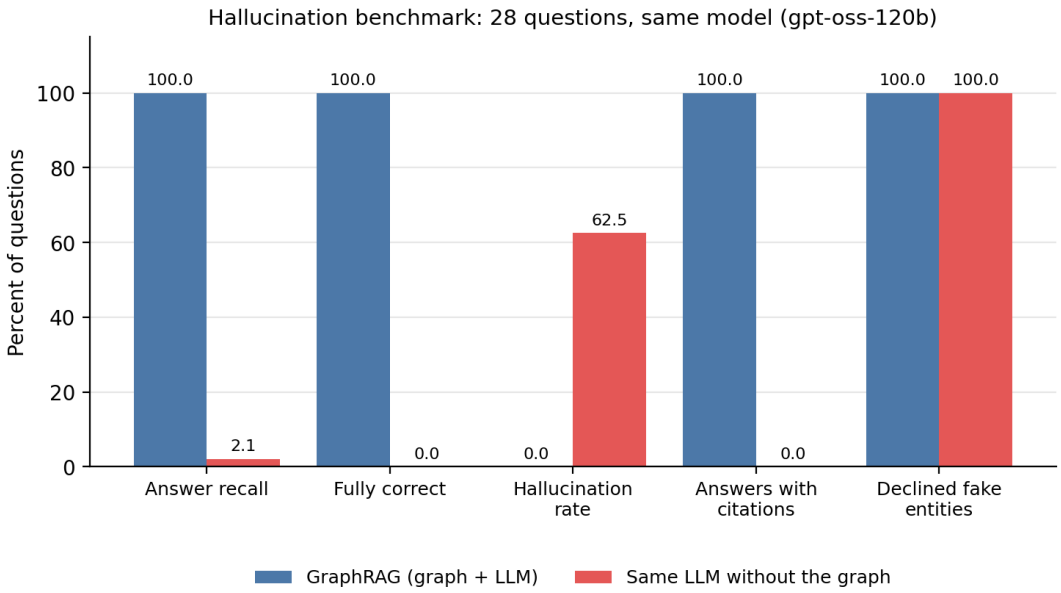


Figure 8. *Hallucination benchmark: GraphRAG vs the same LLM without the graph (28 questions: 24 answerable, 4 about made-up entities).*

Average answer latency was 1.69 s with GraphRAG and 1.50 s without the graph. Citation precision with GraphRAG was 98.3%.

Pipeline performance and reliability (Objective 2, 6)

From CloudWatch over seven days: median 10.94 s per document (p90 19.85 s) for NER + RE and graph writes; 162.7 s on average for a full-text PDF; 2,225 endpoint invocations with 3.53 s mean model latency (both models); 240 API requests with 0 server errors. Four Lambda errors occurred in one rebuild before the throttling fix; all four documents were recovered by Lambda's automatic retry.

User interface and accessibility

The dashboard was tested in Microsoft Edge with Playwright (results/UI_UX_TESTING.md). The landing page loaded in 0.24-0.66 s over three fresh loads, sign-in opened the app in 0.91 s and an answer appeared 2.42 s after asking. A wrong password gives a non-revealing message, non-members see why upload is hidden, the sign-in form works with the keyboard alone, and at phone width (390 x 844) the layout stacks into one column with no horizontal scrolling. An axe-core scan (WCAG 2.0/2.1 A and AA) found no violations in the signed-in app and one on the landing page in the light theme: the password Show button had a contrast ratio of 4.4:1 against the 4.5:1 minimum. The dashboard now uses a single dark theme for every visitor, in which the scan finds no violations (the button's contrast is 6.53:1).

10.3 Baseline Comparison

Table 12. Comparison with baselines under identical conditions

Comparison	Baseline	This project	Conditions
Hallucination rate	Same LLM without the graph: 62.5%	GraphRAG: 0.0%	Same 28 questions, same model (gpt-oss-120b), same scorer
Answer recall	2.1%	100.0%	As above
Relation extraction F1	Earlier RE model: 0.423	0.661	SciERC test, same calibration procedure
Papers with extracted entities	Rule/dictionary extractor: 189/304	SciBERT + dictionary: 305/305	Same corpus
Retrieval latency	44 sequential queries: about 6 s	27 parallel queries: 0.535 s	Same graph and questions
NER F1 (reference)	SciBERT paper on SciERC: about 0.65-0.67 (GPU, tuned)	0.618 (laptop CPU, 4 epochs)	Published result vs our run; different training budget

10.4 Results Summary

Table 13. Objectives, status and evidence

Objective	Success criterion	Result	Status
1. Automated extraction	NER and RE F1 >= 0.60; all papers linked	NER 0.618, RE 0.661; 305/305 papers	Achieved
2. Serverless processing	>= 1,000 docs/hour, no losses	About 1,350 docs/hour, 0 failures	Achieved
3. Grounded answers	Hallucination < 5% and below LLM-only; citations	0.0% vs 62.5%; 100% cited; 98.3% precision	Achieved
4. Graph storage	Retrieval < 1 s; >= 20,000 nodes	0.535 s; 29,320 nodes	Achieved on Neo4j AuraDB; Neptune Analytics configured, not run (Free plan)
5. Interactive dashboard	All features work; 401 without token	T01-T11 pass; no accessibility violations in the app	Achieved
6. Operability	Monitoring, passing end-to-end test, $0 charges	Dashboard, 2 alarms, 13/13 checks, $0	Achieved

11. Discussion, Limitations and Risk Analysis

11.1 Discussion

The main result is the controlled hallucination test. With the same model and the same questions, removing the graph turned 62.5% of answers into confident mistakes (for example, invented authors, venue and year of a survey paper), while GraphRAG answered every answerable question correctly with citations. Both systems refused questions about made-up entities, so the improvement comes from grounding in retrievable facts, not from the model

being more cautious. For users, this means that every statement can be checked by clicking its citation.

The extraction results show that a compact domain model is enough for a useful graph: NER F1 of 0.618 is close to the published SciBERT result despite CPU-only training, and the improved RE model (0.661) adds typed relations that let the system answer "which methods are applied to this task" rather than only "which papers mention it". The serverless design processed about 1,350 documents per hour with no lost documents and costs nothing when idle, which suits bursty literature ingestion.

11.2 Limitations

- **Technical:** the deployed graph and LLM are external services (Neo4j AuraDB, Groq, Gemini) because the Free plan blocks Neptune and Bedrock; the AWS-native path is implemented but was not load-tested. Testing found three defects that are now fixed in the code: the Connect view listed the same path several times (fixed and deployed on 15 October 2026); about 0.1% of graph nodes (37) had been duplicated by concurrent writes (uniqueness constraints are implemented and take effect once the existing duplicates are merged with the clean-up script); and the password Show button was slightly below the WCAG contrast minimum in the light theme, which was removed in favour of a single dark theme.
- **Sample data:** the four sample papers in the repository originally carried placeholder authors and venues, so the live graph could name a placeholder author for a real paper. The sample file now holds the real authors and venues; the old placeholder nodes are removed from the live graph by a one-off clean-up script that backs up every node it changes.
- **Data:** only 306 documents were fully ingested (most of the 20,746 paper nodes are cited references with metadata only); scanned PDFs are not supported (no OCR); English only.
- **Model:** NER F1 for OtherScientificTerm (0.534) and Task (0.576) is lower than for Method (0.707); relation extraction was evaluated with gold entities, so end-to-end relation accuracy on raw text is lower.
- **Evaluation:** the GraphRAG benchmark has 28 questions generated from the graph; it measures grounding and citation quality, not open-ended research questions; no formal user study was run.

- **Security and cost:** MFA is not enforced; secrets are deployment parameters rather than AWS Secrets Manager entries; the free graph pauses after three idle days.

11.3 Risks and Mitigation

Table 14. Risks and mitigation

Risk	Likelihood / impact	Mitigation	Residual risk
Unexpected cloud cost	Low / high (Neptune bills hourly)	Free plan blocks charges; serverless endpoint; Neptune only by parameter; budget alert recommended	Low
Credential exposure / misconfiguration	Medium / high	NoEcho parameters, .gitignore for credential files, least-privilege roles, private encrypted bucket, JWT on all private routes	Low-medium (move to Secrets Manager, enable MFA)
Data quality / availability	Medium / medium	Retraction filter, canonicalisation, OpenAlex enrichment; source data kept in S3; graph rebuildable in about 14 minutes	Low
Provider dependency / service outage	Medium / medium	LLM failover chain ending in cited facts; graph backend switchable to Neptune; free graph resumes on demand	Medium (AuraDB Free pause)
SageMaker throttling during bulk ingestion	Medium / medium	Concurrency 10, exponential back-off with jitter, retry and DLQ	Low
LLM produces unsupported claims	Low / high	Answers restricted to top-30 facts with mandatory citations; benchmark shows 0% hallucination	Low

12. Team Contribution and Project Management

Work was divided equally among the three members by area, following the work distribution in the project proposal. Each member owned one layer of the system end to end (design, implementation, testing and documentation), and all three shared architecture design, integration testing and reviews.

Table 15. Team contribution by activity

Activity	Dhruv Gupta (23BIT0350)	Parth Sharma (23BIT0211)	Devesh Agarwal (23BIT0201)
Literature survey and assigned paper analysis	Papers P3, P4, P9, P12, P13 (KG-grounded generation); lead for the literature survey	Papers P1, P6, P7, P10, P15 (systems and scalability)	Papers P2, P5, P8, P11, P14 (extraction and data)
Research gap and novelty	GraphRAG and hallucination gap; novelty of grounded answers	Cloud deployment, security and scalability gap	Extraction quality and dataset gap
Frontend / application	Graph queries behind Explore and Connect	Dashboard: landing page, sign-in/up, Ask, Explore, Connect, Research 360, uploads; Cytoscape.js graph view	Upload format and live ingestion status

Activity	Dhruv Gupta (23BIT0350)	Parth Sharma (23BIT0211)	Devesh Agarwal (23BIT0201)
Backend / API / database	Graph ontology and schema, graph client and idempotent writer, multi-hop retrieval, ranking, grounded LLM layer	API design and API Lambda (GraphRAG, Explore, Connect, Insights, upload URLs)	Ingest Lambda: parsing, citation resolution, metadata mapping, canonicalisation
Cloud services and security	Neo4j AuraDB / Neptune Analytics configuration; Bedrock / Groq / Gemini integration	SAM template, API Gateway, Cognito, IAM roles, CloudWatch, X-Ray, Amplify	S3 data lake, SageMaker endpoint, S3-to-Lambda event pipeline, dead-letter queue
Dataset / AI-ML	GraphRAG evaluation questions and ground truth	Model deployment checks and endpoint monitoring	OpenAlex and S2ORC acquisition and preprocessing; SciBERT NER and RE training and calibration
Testing and results	GraphRAG hallucination and retrieval benchmarks	End-to-end smoke test, UI/UX and accessibility testing, system metrics	NER/RE evaluation on SciERC, before/after graph comparison, unit tests
Documentation and presentation	Literature survey, research gap analysis, report sections 2-4	Architecture documentation, presentation, report sections 5, 7, 9-11	Dataset documentation, NLP results, report sections 6 and 8

12.1 Individual Contribution Evidence

- **Dhruv Gupta** (knowledge graph and GraphRAG): designed the graph ontology and wrote the graph layer: src/backend/kg/schema.py, graph_client.py, graph_writer.py, retrieval.py and llm.py. He built multi-hop retrieval with ranking and the grounded LLM layer with provider failover, and the GraphRAG benchmarks (scripts/evaluate_graphrag.py; results/GRAPHRAG_EVAL.md, GRAPHRAG_RETRIEVAL_EVAL.md). He led the literature survey and the research gap analysis.
- **Parth Sharma** (dashboard, API and AWS infrastructure): built the dashboard (src/frontend/) and the API Lambda (src/backend/handlers/api.py), defined the AWS infrastructure in template.yaml (API Gateway, Cognito, IAM, CloudWatch dashboard and alarms, Amplify hosting), and wrote the end-to-end test (scripts/smoke_test.py), metrics collection (scripts/collect_metrics.py) and the UI/UX testing report (results/UI_UX_TESTING.md). He prepared the architecture diagrams and presentation material.
- **Devesh Agarwal** (data and NLP extraction): set up the repository and the S3 data lake, wrote the dataset fetchers and parsers (scripts/fetch_openalex.py, fetch_s2orc.py; src/backend/kg/parsers.py, enrich.py, extractors.py, aliases.py) and the ingest Lambda (src/backend/handlers/ingest.py), and trained, calibrated and deployed the SciBERT

NER and RE models (src/ai_model/). He wrote the dataset documentation and the NLP results (results/SCIBERT_RESULTS.md) and reviewed and merged the pull requests.

- **Shared:** system architecture, integration testing on the deployed stack, code reviews through pull requests #1-#6, and the final report.

13. Conclusion and Future Enhancements

13.1 Conclusion

The project addressed the difficulty of extracting and querying knowledge spread across scientific literature, and the tendency of LLMs to hallucinate when asked about it. It delivered a working cloud system on AWS: an event-driven serverless pipeline that extracts entities and typed relations with two SciBERT models on Amazon SageMaker, a knowledge graph of 29,320 nodes, and a secure dashboard that answers questions with GraphRAG and cites its sources. All six objectives were met on the deployed system: extraction F1 of 0.618 (entities) and 0.661 (relations), about 1,350 documents per hour with no failures, 0% hallucination against 62.5% for the same LLM without the graph, 0.535 s retrieval, a fully working dashboard, and 13/13 end-to-end checks at no charge. The original AWS-native graph and LLM services (Neptune Analytics and Bedrock) are implemented as deployment options but were not run because the Free plan blocks them.

13.2 Future Enhancements

- Run the Neptune Analytics and Bedrock configuration on a paid account and compare latency and cost with the free deployment.
- Ingest larger corpora from the OpenAlex and S2ORC snapshots with an AWS Step Functions or AWS Batch backfill, and add OCR (Amazon Textract) for scanned PDFs.
- Train the models on a GPU (once SageMaker training quota is granted) and evaluate end-to-end relation extraction on raw text.
- Move secrets to AWS Secrets Manager, enforce MFA in Cognito, and add AWS WAF rate limiting on the API.
- Extend evaluation with open-ended expert questions and a user study, and add de-duplication of equivalent paths in Connect.

References

Beltagy, I., Lo, K., & Cohan, A. (2019). SciBERT: A pretrained language model for scientific text. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP) (pp. 3613-3618). https://doi.org/10.18653/v1/D19-1371

Chen, C. H., & Shiu, M. F. (2025). KAQG: A knowledge-graph-enhanced RAG for difficulty-controlled question generation (Version 2) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2505.07618

Choi, S., & Jung, Y. (2025). Knowledge graph construction: Extraction, learning, and evaluation. Applied Sciences, 15(7), 3727. https://doi.org/10.3390/app15073727

Dang, F.-R., Tang, J.-T., Pang, K.-Y., Wang, T., Li, S.-S., & Li, X. (2021). Constructing an educational knowledge graph with concepts linked to Wikipedia. Journal of Computer Science and Technology, 36(5), 1200-1211. https://doi.org/10.1007/s11390-020-0328-2

Lin, Q. (2025). Knowledge graph reasoning and completion strategies under few-shot learning. In 2025 International Conference on Artificial Intelligence, Computer, Data Sciences and Applications (ACDSA) (pp. 1-5). IEEE. https://doi.org/10.1109/ACDSA65407.2025.11165842

Lo, K., Wang, L. L., Neumann, M., Kinney, R., & Weld, D. (2020). S2ORC: The Semantic Scholar Open Research Corpus. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (pp. 4969-4983). https://doi.org/10.18653/v1/2020.acl-main.447

Luan, Y., He, L., Ostendorf, M., & Hajishirzi, H. (2018). Multi-task identification of entities, relations, and coreference for scientific knowledge graph construction. In Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (pp. 3219-3232). https://doi.org/10.18653/v1/D18-1360

Mohamed, S., Farah, K., Lotfy, A., Rizk, K., Saeed, A., Mohamed, S., Khouriba, G., & Arafa, T. (2025). Knowledge graphs: The future of data integration and insightful discovery [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2502.15689

Nagy, L., Abonyi, J., & Ruppert, T. (2024). Knowledge graph-based framework to support human-centered collaborative manufacturing in Industry 5.0. Applied Sciences, 14(8), 3398. https://doi.org/10.3390/app14083398

Napoli, R., Morabito, G., Celesti, A., Villari, M., & Fazio, M. (2026). Knowledge graphs-driven intelligence for distributed decision systems [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2602.07614

Pan, S., Luo, L., Wang, Y., Chen, C., Wang, J., & Wu, X. (2024). Unifying large language models and knowledge graphs: A roadmap. IEEE Transactions on Knowledge and Data Engineering, 36(7), 3580-3599. https://doi.org/10.1109/TKDE.2024.3352100

Peng, B., Zhu, Y., Liu, Y., Bo, X., Shi, H., Hong, C., Zhang, Y., & Tang, S. (2025). Graph retrieval-augmented generation: A survey. ACM Transactions on Information Systems, 44(2), 1-52. https://doi.org/10.1145/3777378

Peng, C., Xia, F., Naseriparsa, M., & Osborne, F. (2023). Knowledge graphs: Opportunities and challenges. Artificial Intelligence Review, 56(11), 13071-13102. https://doi.org/10.1007/s10462-023-10465-9

Priem, J., Piwowar, H., & Orr, R. (2022). OpenAlex: A fully-open index of scholarly works, authors, venues, institutions, and concepts [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2205.01833

Sahlab, N., Kahoul, H., Jazdi, N., & Weyrich, M. (2022). A knowledge graph-based method for automating systematic literature reviews. Procedia Computer Science, 207, 2814-2822. https://doi.org/10.1016/j.procs.2022.09.339

Wang, Y., Zhao, X., Tan, Z., Xiao, W., & Cheng, X. (2025). Temporal knowledge graph diffusion model for open-world reasoning. Science China Information Sciences, 68(10), 202101. https://doi.org/10.1007/s11432-024-4329-3

Xu, J., Zhang, H., Zhang, H., Lu, J., & Xiao, G. (2024). ChatTf: A knowledge graph-enhanced intelligent Q&A system for mitigating factuality hallucinations in traditional folklore. IEEE Access, 12, 162638-162650. https://doi.org/10.1109/ACCESS.2024.3485877

Yang, L. (2025). Automated construction method of knowledge graph driven by deep learning. International Journal of Computer Science and Information Technology, 7(1), 1-8. https://doi.org/10.62051/ijcsit.v7n1.01

Zheng, Z., Dong, Y., Wang, S., Liu, H., Wang, Q., & Li, J. (2025). KG-CF: Knowledge graph completion with context filtering under the guidance of large language models [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2501.02711

Zhong, Z., & Chen, D. (2021). A frustratingly easy approach for entity and relation extraction. In Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (pp. 50-61). https://doi.org/10.18653/v1/2021.naacl-main.5

Amazon Web Services. (2026). AWS Serverless Application Model developer guide. https://docs.aws.amazon.com/serverless-application-model/

Amazon Web Services. (2026). Amazon SageMaker serverless inference. https://docs.aws.amazon.com/sagemaker/latest/dg/serverless-endpoints.html

Amazon Web Services. (2026). Amazon Neptune Analytics user guide. https://docs.aws.amazon.com/neptune-analytics/

Neo4j, Inc. (2026). Neo4j AuraDB documentation. https://neo4j.com/docs/aura/

Franz, M., Lopes, C. T., Fong, D., Kucera, M., Cheung, M., Siper, M. C., Huck, G., Dong, Y., Sumer, O., & Bader, G. D. (2023). Cytoscape.js 2023 update: A graph theory library for visualization and analysis. Bioinformatics, 39(1), btad031. https://doi.org/10.1093/bioinformatics/btad031

Appendix A - Project Timeline

Dates are taken from the repository history; the proposal phase preceded the first implementation commit.

Task	Start	End	Owner(s)	Status / evidence
Literature survey	Proposal phase	Proposal submission	All (lead: Dhruv)	Completed: proposal document; literature_survey/
Research gap and novelty	Proposal phase	Proposal submission	All	Completed: proposal; Sections 2-4
Architecture design and repository setup	31/07/2026	12/10/2026	All	Completed: initial commit 5095372; template.yaml
Implementation and integration	12/10/2026	14/10/2026	All	Completed: PR #1-#3 merged 14/10/2026
Proposal review and improvements	14/10/2026	15/10/2026	All	Completed: PR #4, PR #5 merged
Testing and report	14/10/2026	15/10/2026	All	Completed: results/, smoke test 13/13, this report

Task	Start	End	Owner(s)	Status / evidence
UI/UX testing, fixes and redeployment	15/10/2026	15/10/2026	All (UI/UX: Parth)	Completed: PR #6; backend redeployed, smoke test 13/13

Appendix B - Final Review Checklist

☑ Abstract is 200-300 words.

☑ 15 research papers are reviewed and cited.

☑ Each student has completed an independent research gap analysis for assigned papers.

☑ 4-6 measurable objectives are stated.

☑ Novelty is clearly differentiated from existing work and supported by evidence.

☑ Both cloud deployment and complete system architecture diagrams are included.

☑ Dataset/input source, size, format, license, purpose, and preprocessing are documented where applicable.

☑ Cloud provider, services, purpose, status, security, monitoring, and estimated cost are documented.

☑ Implementation evidence and test cases are included; planned work is distinguished from completed work.

☑ Results are genuine, reproducible where possible, and mapped to objectives.

☑ Repository, version-control workflow, and individual contributions are documented.

☑ APA references and in-text citations are consistent.

☑ No credentials or confidential data appear in the report or repository.

☑ Required formatting is applied and placeholders have been removed.